

Curso Inicial Front-End

Clase 17: El DOM:

Selección y

Manipulación

Módulo 2: Dominio de JavaScript y Lógica de Programación



Presentación

¿Cómo hace Facebook para cambiar el color de un botón o mostrar un mensaje nuevo sin recargar la página? En esta clase conoceremos el **DOM (Document Object Model)**, la interfaz que permite que **JavaScript** tome el control de tu **HTML**. Aprenderemos a seleccionar elementos, modificar su contenido y transformar la estructura de la web en tiempo real.



Objetivos

- Comprender qué es el DOM y cómo **JavaScript** interactúa con el árbol de nodos.
- Dominar los selectores modernos (`querySelector`, `querySelectorAll`) frente a los tradicionales.
- Manipular el contenido, los estilos y las clases **CSS** de los elementos de forma dinámica.



Bloques Temáticos

- Tema 1: Árbol de nodos y selección de elementos (querySelector).
 - ¿Qué es el DOM? El Control Remoto de la Web
 - Seleccionando Nodos: El Lazo Mágico
- Tema 2: Modificación dinámica de contenido e interactividad básica.
 - Modificar el Contenido Textual
 - Manipular Clases: El Guardarropa Dinámico
 - Crear Elementos desde la Nada (El Modo Dios)

En las clases anteriores aprendimos lógica pura (arrays, objetos, variables). Todo eso vivía oculto en la consola matemática. Pero los usuarios de tu página web no miran la consola; ellos miran la pantalla interactiva.

Hoy aprenderemos cómo hacer que nuestra lógica matemática "cruce el puente" y empiece a inyectar, borrar y modificar cajas **HTML** y textos reales directamente frente a los ojos del usuario.

 **Recurso Oficial:** [MDN Web Docs - Introducción al DOM](#)

Tema 1: Árbol de nodos y selección de elementos (querySelector).

1. ¿Qué es el DOM? El Control Remoto de la Web

El **DOM (Document Object Model)** es la herramienta mágica que el navegador nos regala para conectar **JavaScript** con **HTML**.

- El **HTML**: Es el **Televisor**. Muestra lo que está programado y no puede cambiar por sí solo.
- El **DOM**: Es el **Control Remoto**. Es una réplica exacta del **HTML**, pero en forma de "Objetos de **JS**". **JavaScript** agarra este control remoto para subir el volumen, cambiar de canal (texto) o apagar la pantalla (borrar botones) de forma dinámica.

En lenguaje simple: El DOM es el mapa que el navegador crea para entender tu **HTML**. Cada vez que cambias algo con **JS** (como el color de un botón), el navegador tiene que recalcular dónde va cada cosa (Reflow) y volver a dibujarla (Repaint).

Traducción Técnica: El DOM es una **API orientada a objetos** para documentos **HTML/XML**. Representa el documento como un grafo de nodos (objetos vivos en memoria) que heredan de las interfaces **Node** y **Element**. **JavaScript** interactúa con estos objetos a través del motor de renderizado. Cualquier modificación del DOM puede disparar procesos de **Reflow** (re-cálculo

de layout) y **Repaint** (dibujado), afectando directamente el rendimiento de la interfaz.

El Árbol Genealógico

El navegador lee tu archivo `.html` y lo transforma en un árbol genealógico invertido en su memoria:

- `<html>` es la Raíz (El abuelo de todos).
- `<body>` es un Nodo Padre.
- `<h1>` o `<img>` son los Nodos Hijos.

2. Seleccionando Nodos: El Lazo Mágico

Para que el control remoto funcione, primero tienes que decirle a **JS** a qué botón específico del televisor le quieres apuntar. Esto se llama **Selección del DOM**.

JavaScript nos da el objeto global `document` (el control remoto en sí) y varios métodos para buscar.

Comparativa: Herramientas de Selección

Selector en JS	Mapeo Visual (¿Qué atrapo?)	¿Qué me devuelve?	¿Cuándo usarlo?
<code>.getElementById("logo")</code>	Busca estrictamente el <code>id="logo"</code> . (Fíjate que NO se usa el	Un solo elemento HTML exacto.	Cuando sabes el ID. Es rapidísimo.

Selector en <u>JS</u>	Mapeo Visual (¿Qué atrapo?)	¿Qué me devuelve?	¿Cuándo usarlo?
	numeral # aquí adentro).		
<code>.querySelector(".tarjeta")</code>	La navaja suiza. Adentro debes escribir sintaxis CSS exacta (usas el punto . para clases, y # para IDs).	Solo el primero que se cruce en su camino, e ignora a los demás.	El 90% de las veces. Es el estándar moderno.
<code>.querySelectorAll(".tarjeta")</code>	Tira una red gigante sobre toda la página.	Una Lista (Array) con todas las tarjetas encontradas.	Cuando quieres borrar o modificar 50 cosas al mismo tiempo.

Tema 2: Modificación dinámica de contenido e interactividad básica.

1. Modificar el Contenido Textual

¡Lo atrapaste! Tienes tu botón o tu título guardado en una variable (`const titulo = document.querySelector("#tituloPrincipal")`). Ahora vamos a hackearlo.

Comparativa de Textos: Cuidado con la Seguridad

Propiedad	¿Qué inyecta en la pantalla?	Nivel de Peligro
<code>.textContent</code>	Inyecta Texto Plano . Ignora cualquier etiqueta. Si le pides inyectar <code>Hola</code> , mostrará literalmente los corchetes feos en pantalla.	Nulo (100% Seguro) . Bloquea automáticamente intentos de hackeo. Úsalo siempre por defecto.
<code>.innerHTML</code>	Inyecta código y obliga al navegador a renderizarlo. Si pasas <code>Hola</code> , la palabra Hola aparecerá en negritas.	Riesgo Crítico . Si este texto proviene de la escritura de un usuario, un hacker puede inyectarte código malicioso <code><script></code> y robarte datos (Vulnerabilidad XSS).

Propiedad	¿Qué inyecta en la pantalla?	Nivel de Peligro
<code>.value</code>	Exclusivo para leer o escribir lo que el usuario tipea adentro de campos de formulario (<code><input></code> , <code><select></code>).	Nulo.

🔴 **Checkpoint de Comprensión:** Tienes una caja de comentarios en tu blog. Decides usar `cajaFondo.innerHTML = inputUsuario.value` para agarrar lo que el usuario tecleó y dibujarlo en la pantalla. Un usuario malicioso teclaea exactamente este texto: `<script> alert("Virus instalado") </script>`. Si usaste `innerHTML`, ¿qué crees que pasará cuando otro usuario inocente abra la página web para leer los comentarios? (Pista: Piensa en si el navegador leerá el comentario como un texto tonto o como una orden de ejecución real).

2. Manipular Clases: El Guardarropa Dinámico

Puedes usar **JS** para inyectar **CSS** en línea (`titulo.style.color = "red"`), pero en la industria profesional eso se considera una pésima práctica (Código Sucio).

La forma correcta es tener tus diseños pre-armados en tu archivo `styles.css` como clases (ej: `.alerta-roja { color: red }`), y decirle a **JS** que le "ponga o le quite esa ropa" al elemento usando la propiedad `.classList`.

- `.classList.add("alerta-roja")`: Le pone la remera roja.
- `.classList.remove("alerta-roja")`: Le quita la remera roja.

- `.classList.toggle("alerta-roja")`: Es un interruptor inteligente. Si tiene la remera, se la quita; si no la tiene, se la pone.
- `.classList.contains("alerta-roja")`: ¿Tiene puesta la remera? (Te responde con un `true` o `false` matemático).

La batalla de los Selectores: ¿Cuál elegir?

Selector	Velocidad	Flexibilidad	Uso
<code>getElementById</code>	 Máxima	 Baja	Para IDs únicos (id="login").
<code>querySelector</code>	 Media	 Altísima	Usa sintaxis CSS (#id, .clase, tag).
<code>querySelectorAll</code>	 Media	 Altísima	Devuelve una lista (NodeList) de todos.

Modificando Formularios: La propiedad `.checked`

Cuando trabajamos con formularios, no solo nos importa el texto (`.value`). Si tienes un **Checkbox** o un **Radio Button**, para saber si está marcado usamos `.checked`:

```
const terminos = document.querySelector("#terminos");
if (terminos.checked) {
  console.log("El usuario aceptó los términos.");
}
```

Casos de Uso del DOM en la vida real

- **Validación en vivo**: Avisar al usuario que su contraseña es corta mientras escribe.

- **Modo Oscuro:** Cambiar las clases CSS del body al presionar un interruptor.
- **Carrito de Compras:** Sumar productos a una lista visual sin recargar la página.

3. Crear Elementos desde la Nada (El Modo Dios)

Podemos hacer que aparezcan bloques HTML desde cero sin que la página se tenga que recargar. Es un proceso estricto de 3 pasos: Crear, Vestir, y Aterrizar.

```
/* PASO 1: CREAR (Flotando en la nada) */
// Le decimos a JS que invente una viñeta <li> en la memoria RAM. Todavía
no existe en la pantalla.
const nuevaViñeta = document.createElement("li");

/* PASO 2: VESTIR (Asignar datos) */
// Le inyectamos texto y le ponemos una clase CSS de tu archivo de estilos
nuevaViñeta.textContent = "Nueva Tarea de prueba";
nuevaViñeta.classList.add("item-fuerte");

/* PASO 3: ATERRIZAR (Inyección final) */
// Buscamos a un "Padre" en la pantalla (ej: una etiqueta <ul>), y le
pegamos al hijo recién creado.
const listaPadre = document.querySelector("#lista-tareas");
listaPadre.appendChild(nuevaViñeta); // ¡Bum! Aparece mágicamente en el
monitor del usuario.

/* PASO 4 (Bonus): BORRAR */
// Enviar un elemento de vuelta a la nada es tan fácil como llamar a
.remove()
// nuevaViñeta.remove();
```

Documentación Oficial (Referencias)

- **CSS:** <https://developer.mozilla.org/es/docs/Web/CSS>
- **HTML:** <https://developer.mozilla.org/es/docs/Web/HTML>
- **JS:** <https://developer.mozilla.org/es/docs/Web/JavaScript>
- **JavaScript:** <https://developer.mozilla.org/es/docs/Web/JavaScript>